



UNSW
SYDNEY

COMP3153/9153

Algorithmic Verification

Lecture 2: Temporal Logics

Intersection of Languages

Problem

Let A be a FA such that $\mathcal{L}(A)$ is the set of strings with an even number of as.

Intersection of Languages

Problem

Let A be a FA such that $\mathcal{L}(A)$ is the set of strings with an even number of as.

Let B be a FA such that $\mathcal{L}(B)$ is the set of strings with an odd number of bs.

Intersection of Languages

Problem

Let A be a FA such that $\mathcal{L}(A)$ is the set of strings with an even number of as.

Let B be a FA such that $\mathcal{L}(B)$ is the set of strings with an odd number of bs.

How can we **combine** A and B into a new automata C such that $\mathcal{L}(C) = \mathcal{L}(A) \cap \mathcal{L}(B)$?

(try to come up with a general technique for any automata)

Intersection of Languages

Problem

Let A be a FA such that $\mathcal{L}(A)$ is the set of strings with an even number of as.

Let B be a FA such that $\mathcal{L}(B)$ is the set of strings with an odd number of bs.

How can we **combine** A and B into a new automata C such that $\mathcal{L}(C) = \mathcal{L}(A) \cap \mathcal{L}(B)$?

(try to come up with a general technique for any automata)

We need to create the **product** of two automata.

Automata Product

Definition

The **product** of two automata

$$A_1 = (Q_1, q_0^1, \Sigma_1, \delta_1, F_1) \text{ and}$$

$$A_2 = (Q_2, q_0^2, \Sigma_2, \delta_2, F_2)$$

is defined as: $(Q, q_0, \Sigma, \delta, F)$ where:

Automata Product

Definition

The **product** of two automata

$$A_1 = (Q_1, q_0^1, \Sigma_1, \delta_1, F_1) \text{ and}$$

$$A_2 = (Q_2, q_0^2, \Sigma_2, \delta_2, F_2)$$

is defined as: $(Q, q_0, \Sigma, \delta, F)$ where:

- $Q = Q_1 \times Q_2$

Automata Product

Definition

The **product** of two automata

$$A_1 = (Q_1, q_0^1, \Sigma_1, \delta_1, F_1) \text{ and}$$

$$A_2 = (Q_2, q_0^2, \Sigma_2, \delta_2, F_2)$$

is defined as: $(Q, q_0, \Sigma, \delta, F)$ where:

- $Q = Q_1 \times Q_2$
- $q_0 = (q_0^1, q_0^2)$

Automata Product

Definition

The **product** of two automata

$$A_1 = (Q_1, q_0^1, \Sigma_1, \delta_1, F_1) \text{ and}$$

$$A_2 = (Q_2, q_0^2, \Sigma_2, \delta_2, F_2)$$

is defined as: $(Q, q_0, \Sigma, \delta, F)$ where:

- $Q = Q_1 \times Q_2$
- $q_0 = (q_0^1, q_0^2)$
- $\Sigma = \Sigma_1 \cup \Sigma_2$

Automata Product

Definition

The **product** of two automata

$$A_1 = (Q_1, q_0^1, \Sigma_1, \delta_1, F_1) \text{ and}$$

$$A_2 = (Q_2, q_0^2, \Sigma_2, \delta_2, F_2)$$

is defined as: $(Q, q_0, \Sigma, \delta, F)$ where:

- $Q = Q_1 \times Q_2$
- $q_0 = (q_0^1, q_0^2)$
- $\Sigma = \Sigma_1 \cup \Sigma_2$
- $\delta((q_1, q_2), a) =$

$\{(q'_1, q'_2) \mid q'_1 \in \delta_1(q_1, a), q'_2 \in \delta_2(q_2, a)\}$	if $a \in \Sigma_1 \cap \Sigma_2$
$\{(q'_1, q_2) \mid q'_1 \in \delta_1(q_1, a)\}$	if $a \in \Sigma_1 \setminus \Sigma_2$
$\{(q_1, q'_2) \mid q'_2 \in \delta_2(q_2, a)\}$	if $a \in \Sigma_2 \setminus \Sigma_1$

Automata Product

Definition

The **product** of two automata

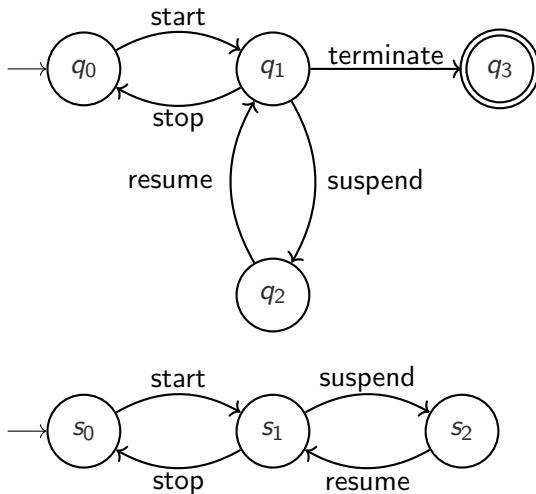
$$A_1 = (Q_1, q_0^1, \Sigma_1, \delta_1, F_1) \text{ and}$$

$$A_2 = (Q_2, q_0^2, \Sigma_2, \delta_2, F_2)$$

is defined as: $(Q, q_0, \Sigma, \delta, F)$ where:

- $Q = Q_1 \times Q_2$
- $q_0 = (q_0^1, q_0^2)$
- $\Sigma = \Sigma_1 \cup \Sigma_2$
- $\delta((q_1, q_2), a) =$
$$\begin{cases} \{(q'_1, q'_2) \mid q'_1 \in \delta_1(q_1, a), q'_2 \in \delta_2(q_2, a)\} & \text{if } a \in \Sigma_1 \cap \Sigma_2 \\ \{(q'_1, q_2) \mid q'_1 \in \delta_1(q_1, a)\} & \text{if } a \in \Sigma_1 \setminus \Sigma_2 \\ \{(q_1, q'_2) \mid q'_2 \in \delta_2(q_2, a)\} & \text{if } a \in \Sigma_2 \setminus \Sigma_1 \end{cases}$$
- $F = F_1 \times F_2$

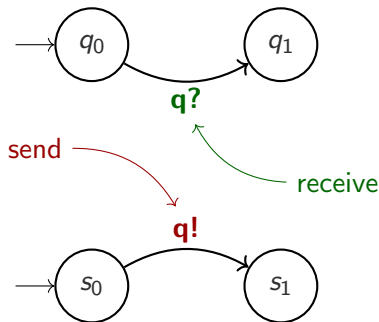
Task and Scheduler



Products can encode **communication**. Compute the product of these two processes.

Message passing

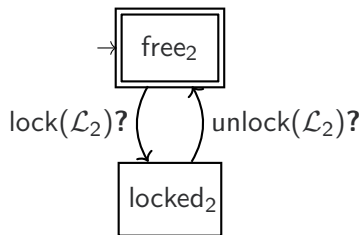
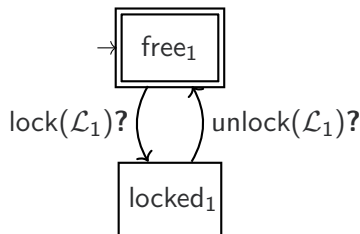
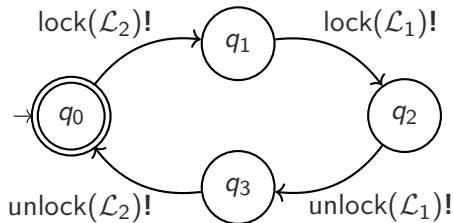
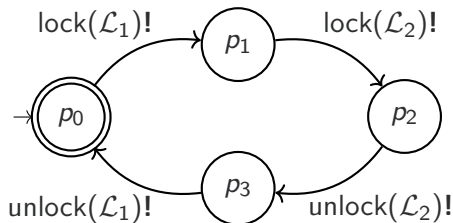
It is often useful (e.g. more efficient) to keep the product implicit and describe only the individual components. Components synchronize via **message passing** which may be one-to-one (**unicast**) or one-to-many (**broadcast**).



Different tools offer **broadcast** or **unicast** communication. **Check the manual!**

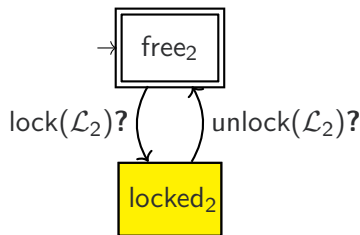
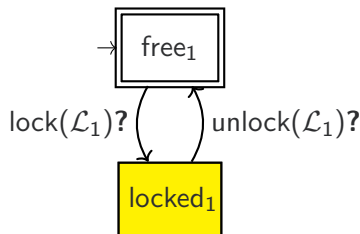
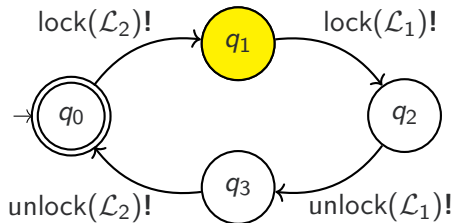
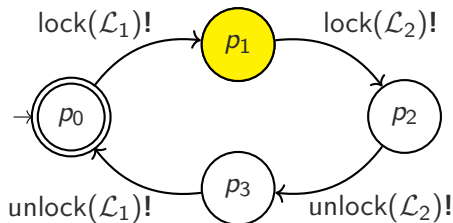
Deadlock Example

Assuming **unicast** synchronisation:



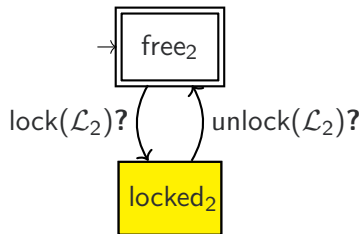
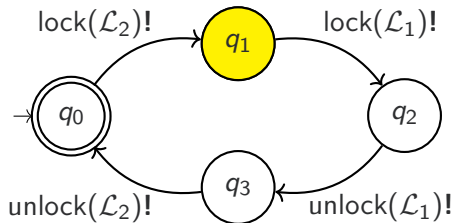
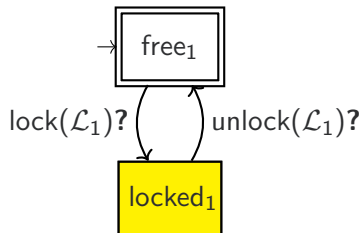
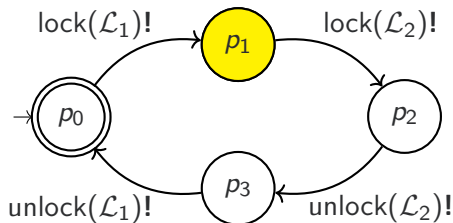
Deadlock Example

Assuming **unicast** synchronisation:



Deadlock Example

Assuming **unicast** synchronisation:



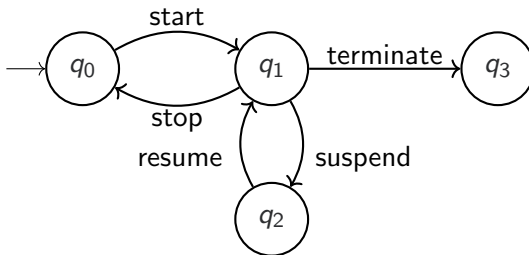
Deadlock is a state (in the product) which has **no outgoing transitions**

Kripke Structures

Definition

A *labelled automaton* is a FA $(Q, q_0, \Sigma, \delta, F, L)$ with an additional labelling function $L : Q \rightarrow 2^{\mathcal{P}}$, where $\mathcal{P}$ is our *atomic propositions*.

A *Kripke structure* is a type of labelled automaton where $|\Sigma| = 1$, $F = Q$. Equivalently, we don't have a notion of actions or final states, and $\delta : Q \rightarrow 2^Q$. We also require that for any q , $\delta(q) \neq \emptyset$.

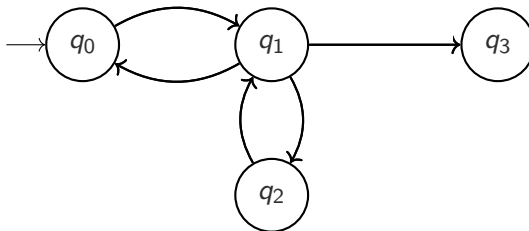


Kripke Structures

Definition

A *labelled automaton* is a FA $(Q, q_0, \Sigma, \delta, F, L)$ with an additional labelling function $L : Q \rightarrow 2^{\mathcal{P}}$, where $\mathcal{P}$ is our *atomic propositions*.

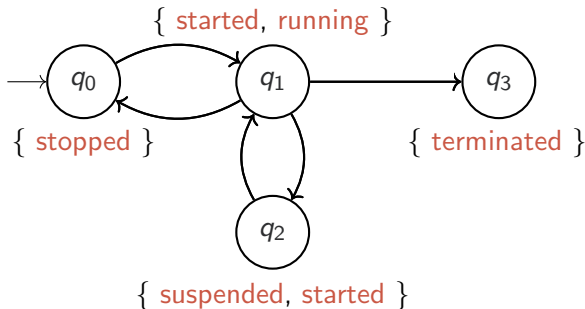
A *Kripke structure* is a type of labelled automaton where $|\Sigma| = 1$, $F = Q$. Equivalently, we don't have a notion of actions or final states, and $\delta : Q \rightarrow 2^Q$. We also require that for any q , $\delta(q) \neq \emptyset$.



Kripke Structures

Definition

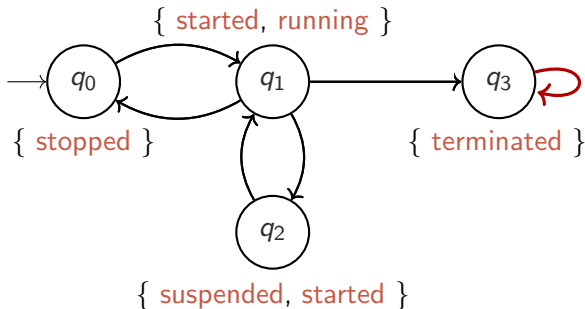
A *labelled automaton* is a FA $(Q, q_0, \Sigma, \delta, F, L)$ with an additional labelling function $L : Q \rightarrow 2^{\mathcal{P}}$, where $\mathcal{P}$ is our *atomic propositions*. A *Kripke structure* is a type of labelled automaton where $|\Sigma| = 1$, $F = Q$. Equivalently, we don't have a notion of actions or final states, and $\delta : Q \rightarrow 2^Q$. We also require that for any q , $\delta(q) \neq \emptyset$.



Kripke Structures

Definition

A *labelled automaton* is a FA $(Q, q_0, \Sigma, \delta, F, L)$ with an additional labelling function $L : Q \rightarrow 2^{\mathcal{P}}$, where $\mathcal{P}$ is our *atomic propositions*. A *Kripke structure* is a type of labelled automaton where $|\Sigma| = 1$, $F = Q$. Equivalently, we don't have a notion of actions or final states, and $\delta : Q \rightarrow 2^Q$. We also require that for any q , $\delta(q) \neq \emptyset$.



Traces

Definition

A *trace*, also called a *behaviour*, is the sequence of labels corresponding to a run. For Kripke structures it is necessarily infinite in length.

Define $\text{Traces}(A)$ to be all possible infinite traces from q_0 in A .

Definition

A linear time *property* is a set of traces, i.e. a subset of $(2^P)^\omega$. We say a Kripke structure A *satisfies* a property P iff:

$$\text{Traces}(A) \subseteq P$$

LTL

Linear temporal logic (LTL) is a *logic* designed to describe linear time properties.

Linear temporal logic syntax

We have normal *propositional operators*:

- $p \in \mathcal{P}$ is an LTL formula.
- If φ, ψ are LTL formulae, then $\varphi \wedge \psi$ is an LTL formula.
- If φ is an LTL formula, $\neg\varphi$ is an LTL formula.

LTL

Linear temporal logic (LTL) is a *logic* designed to describe linear time properties.

Linear temporal logic syntax

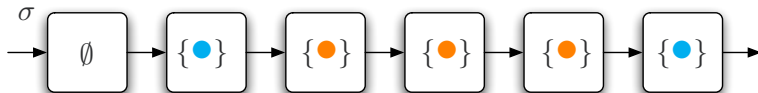
We have normal **propositional operators**:

- $p \in \mathcal{P}$ is an LTL formula.
- If φ, ψ are LTL formulae, then $\varphi \wedge \psi$ is an LTL formula.
- If φ is an LTL formula, $\neg\varphi$ is an LTL formula.

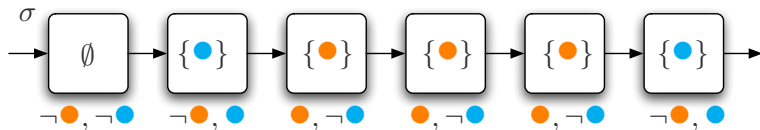
We also have **modal** or **temporal operators**:

- If φ is an LTL formula, then $\mathbf{X} \varphi$ is an LTL formula.
- If φ, ψ are LTL formulae, then $\varphi \mathbf{U} \psi$ is an LTL formula.

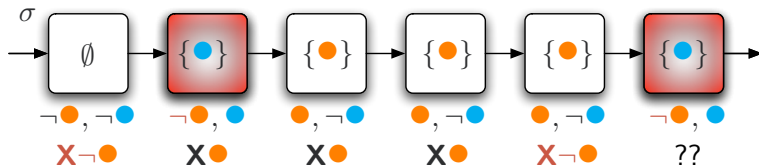
LTL Semantics in Pictures



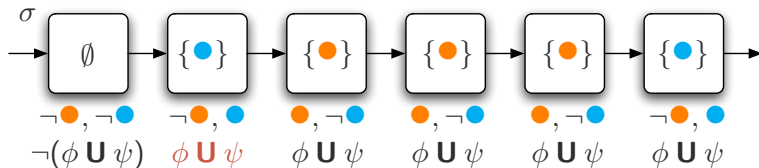
LTL Semantics in Pictures



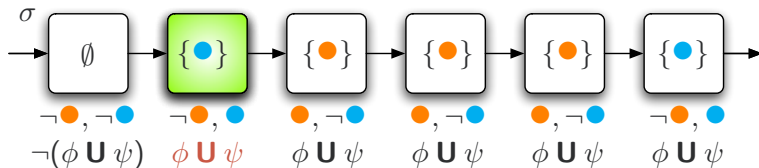
LTL Semantics in Pictures



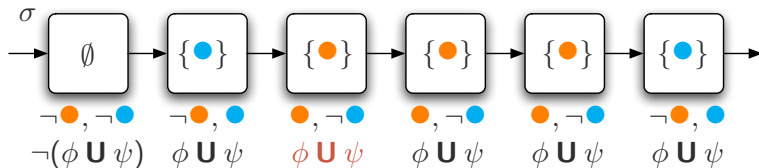
LTL Semantics in Pictures



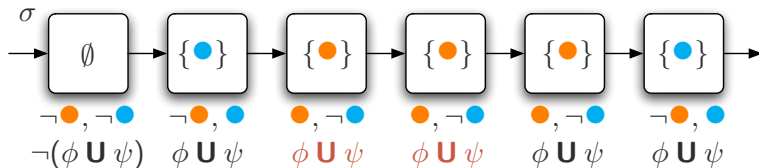
LTL Semantics in Pictures



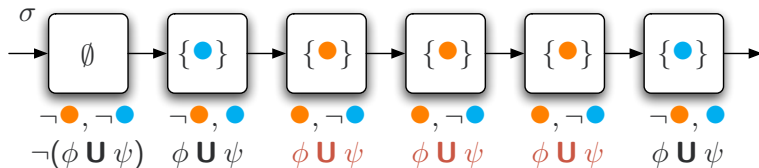
LTL Semantics in Pictures



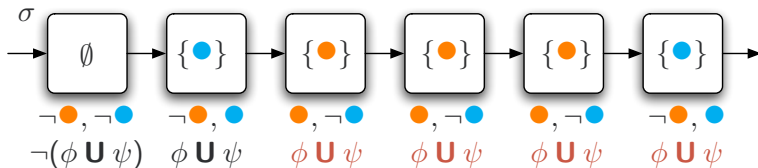
LTL Semantics in Pictures



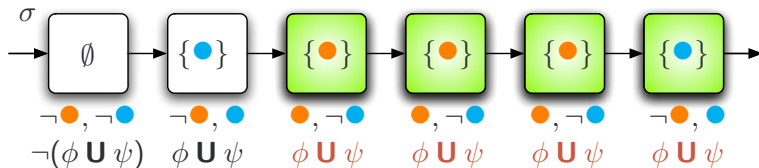
LTL Semantics in Pictures



LTL Semantics in Pictures



LTL Semantics in Pictures



LTL Semantics

Let $\sigma = \sigma_0\sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$ be a trace. Then define notation:

- $\sigma|_0 = \sigma$
- $\sigma|_1 = \sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$
- $\sigma|_{n+1} = (\sigma|_1)|_n$

LTL Semantics

Let $\sigma = \sigma_0\sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$ be a trace. Then define notation:

- $\sigma|_0 = \sigma$
- $\sigma|_1 = \sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$
- $\sigma|_{n+1} = (\sigma|_1)|_n$

Semantics

The models of LTL are **traces**. For atomic propositions, we just look at the **first state**:

$$\sigma \models p \quad \Leftrightarrow \quad p \in \sigma_0$$

LTL Semantics

Let $\sigma = \sigma_0\sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$ be a trace. Then define notation:

- $\sigma|_0 = \sigma$
- $\sigma|_1 = \sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$
- $\sigma|_{n+1} = (\sigma|_1)|_n$

Semantics

The models of LTL are **traces**. For atomic propositions, we just look at the **first state**:

$$\sigma \models p \quad \Leftrightarrow \quad p \in \sigma_0$$

$$\sigma \models \varphi \wedge \psi \quad \Leftrightarrow \quad \sigma \models \varphi \text{ and } \sigma \models \psi$$

$$\sigma \models \neg \varphi \quad \Leftrightarrow \quad \sigma \not\models \varphi$$

LTL Semantics

Let $\sigma = \sigma_0\sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$ be a trace. Then define notation:

- $\sigma|_0 = \sigma$
- $\sigma|_1 = \sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$
- $\sigma|_{n+1} = (\sigma|_1)|_n$

Semantics

The models of LTL are **traces**. For atomic propositions, we just look at the **first state**:

$$\begin{array}{ll} \sigma \models p & \Leftrightarrow p \in \sigma_0 \\ \sigma \models \varphi \wedge \psi & \Leftrightarrow \sigma \models \varphi \text{ and } \sigma \models \psi \\ \sigma \models \neg \varphi & \Leftrightarrow \sigma \not\models \varphi \\ \sigma \models \mathbf{X} \varphi & \Leftrightarrow \sigma|_1 \models \varphi \end{array}$$

LTL Semantics

Let $\sigma = \sigma_0\sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$ be a trace. Then define notation:

- $\sigma|_0 = \sigma$
- $\sigma|_1 = \sigma_1\sigma_2\sigma_3\sigma_4\sigma_5 \dots$
- $\sigma|_{n+1} = (\sigma|_1)|_n$

Semantics

The models of LTL are **traces**. For atomic propositions, we just look at the **first state**:

$$\begin{aligned}
 \sigma \models p & \Leftrightarrow p \in \sigma_0 \\
 \sigma \models \varphi \wedge \psi & \Leftrightarrow \sigma \models \varphi \text{ and } \sigma \models \psi \\
 \sigma \models \neg \varphi & \Leftrightarrow \sigma \not\models \varphi \\
 \sigma \models \mathbf{X} \varphi & \Leftrightarrow \sigma|_1 \models \varphi \\
 \sigma \models \varphi \mathbf{U} \psi & \Leftrightarrow \text{There exists an } i \text{ such that } \sigma|_i \models \psi \\
 & \text{and for all } j < i, \sigma|_j \models \varphi
 \end{aligned}$$

We say $A \models \varphi$ iff $\forall \sigma \in \text{Traces}(A). \sigma \models \varphi$.

Derived Operators

The operator **F** φ (“finally” or “eventually”) says that φ will be true at some point.

The operator **G** φ (“globally” or “always”) says that φ is always true.

Exercise

- Give the semantics of **F** and **G**.
- Define **F** and **G** in terms of other operators.

More Exercises

Let ρ be this trace:



$\rho \models \text{orange?}$

More Exercises

Let ρ be this trace:



$$\rho \models \text{orange?}$$

$$\rho \models \text{blue?}$$

More Exercises

Let ρ be this trace:



$$\rho \models \text{orange?}$$

$$\rho \models \text{blue?}$$

$$\rho \models \mathbf{X} \text{ blue?}$$

More Exercises

Let ρ be this trace:



$$\rho \models \text{orange?}$$

$$\rho \models \text{blue?}$$

$$\rho \models \mathbf{X} \text{ blue?}$$

$$\rho \models \mathbf{F} \text{ orange?}$$

More Exercises

Let ρ be this trace:



$$\rho \models \text{orange?}$$

$$\rho \models \text{blue?}$$

$$\rho \models \mathbf{X} \text{ blue?}$$

$$\rho \models \mathbf{F} \text{ orange?}$$

$$\rho|_3 \models \mathbf{F} (\text{orange} \wedge \neg \text{blue})?$$

More Exercises

Let ρ be this trace:



$$\rho \models \text{orange?}$$

$$\rho \models \text{blue?}$$

$$\rho \models \mathbf{X} \text{ blue?}$$

$$\rho \models \mathbf{F} \text{ orange?}$$

$$\rho|_3 \models \mathbf{F} (\text{orange} \wedge \neg \text{blue})?$$

$$\rho \models \mathbf{FG} (\text{orange} \wedge \text{blue})?$$

More Exercises

Let ρ be this trace:



$$\rho \models \text{orange?}$$

$$\rho \models \text{blue?}$$

$$\rho \models \mathbf{X} \text{ blue?}$$

$$\rho \models \mathbf{F} \text{ orange?}$$

$$\rho|_3 \models \mathbf{F} (\text{orange} \wedge \neg \text{blue})?$$

$$\rho \models \mathbf{FG} (\text{orange} \wedge \text{blue})?$$

$$\rho \models \mathbf{G} (\text{orange} \text{ U } \text{blue})?$$

More Exercises

Let ρ be this trace:



More Derived Operators

- Define “Infinitely Often” in LTL.
- Define “Almost Globally” in LTL (always true from some point onwards).

$$\rho \models \text{orange?}$$

$$\rho \models \text{blue?}$$

$$\rho \models \mathbf{X} \text{ blue?}$$

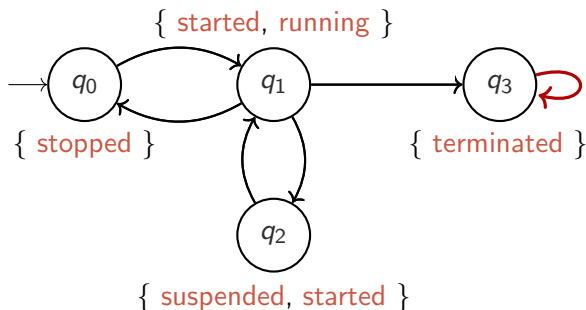
$$\rho \models \mathbf{F} \text{ orange?}$$

$$\rho|_3 \models \mathbf{F} (\text{orange} \wedge \neg \text{blue})?$$

$$\rho \models \mathbf{FG} (\text{orange} \wedge \text{blue})?$$

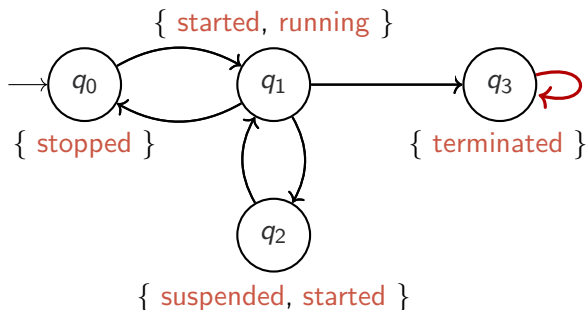
$$\rho \models \mathbf{G} (\text{orange} \mathbf{U} \text{blue})?$$

Possible Futures



We can see that it is **always possible** for a run to move to the **terminated** state. How do we express this in LTL?

Possible Futures



We can see that it is **always possible** for a run to move to the **terminated** state. How do we express this in LTL? **We can't!** — it is a **branching time** property.

Branching Time

Definition

The *computation tree* of a Kripke structure A , written $\text{Tree}(A)$, is an infinite tree of Kripke structure states, where q_0 is the root and a state q' is a child of q if $q' \in \delta(q)$.

Branching Time

Definition

The *computation tree* of a Kripke structure A , written $\text{Tree}(A)$, is an infinite tree of Kripke structure states, where q_0 is the root and a state q' is a child of q if $q' \in \delta(q)$.

A *path* $t_1 t_2 t_3 \dots$ is a (infinite) sequence of computation trees such that t_{n+1} is the child of t_n . Define $\text{Paths}(t)$ to be the set of all paths starting at t .

Branching Time

Definition

The *computation tree* of a Kripke structure A , written $\text{Tree}(A)$, is an infinite tree of Kripke structure states, where q_0 is the root and a state q' is a child of q if $q' \in \delta(q)$.

A *path* $t_1 t_2 t_3 \dots$ is a (infinite) sequence of computation trees such that t_{n+1} is the child of t_n . Define $\text{Paths}(t)$ to be the set of all paths starting at t .

Exercise

Draw the CT for the process example.

CTL* Syntax

Definition

We define two types of formulae, **state formulae** and **path formulae**, named based on their models.

A state formula (SF) is defined as follows:

- All $p \in \mathcal{P}$ are SFs.
- Given SFs P and Q , $\neg P$ is a SF and $P \wedge Q$ is a SF.

CTL* Syntax

Definition

We define two types of formulae, **state formulae** and **path formulae**, named based on their models.

A state formula (SF) is defined as follows:

- All $p \in \mathcal{P}$ are SFs.
- Given SFs P and Q , $\neg P$ is a SF and $P \wedge Q$ is a SF.
- Given a **PF** φ , **E** φ and **A** φ are SFs.

CTL* Syntax

Definition

We define two types of formulae, **state formulae** and **path formulae**, named based on their models.

A state formula (SF) is defined as follows:

- All $p \in \mathcal{P}$ are SFs.
- Given SFs P and Q , $\neg P$ is a SF and $P \wedge Q$ is a SF.
- Given a **PF** φ , **E** φ and **A** φ are SFs.

A path formula (PF) is defined much like LTL:

- If P is a SF, then P is a PF.

CTL* Syntax

Definition

We define two types of formulae, **state formulae** and **path formulae**, named based on their models.

A state formula (SF) is defined as follows:

- All $p \in \mathcal{P}$ are SFs.
- Given SFs P and Q , $\neg P$ is a SF and $P \wedge Q$ is a SF.
- Given a **PF** φ , **E** φ and **A** φ are SFs.

A path formula (PF) is defined much like LTL:

- If P is a SF, then P is a PF.
- Given PFs φ and ψ , $\neg\varphi$ is a PF and $\varphi \wedge \psi$ is a PF.

CTL* Syntax

Definition

We define two types of formulae, **state formulae** and **path formulae**, named based on their models.

A state formula (SF) is defined as follows:

- All $p \in \mathcal{P}$ are SFs.
- Given SFs P and Q , $\neg P$ is a SF and $P \wedge Q$ is a SF.
- Given a **PF** φ , **E** φ and **A** φ are SFs.

A path formula (PF) is defined much like LTL:

- If P is a SF, then P is a PF.
- Given PFs φ and ψ , $\neg\varphi$ is a PF and $\varphi \wedge \psi$ is a PF.
- Given a PF φ then **X** φ is a PF.
- Given PFs φ and ψ , φ **U** ψ is a PF.

Initially, we start with **state formulae** (SFs).

CTL* Semantics

State Semantics

$$t \models p \quad \Leftrightarrow \quad p \in L(t_{\text{root}})$$

CTL* Semantics

State Semantics

$$t \models p \quad \Leftrightarrow \quad p \in L(t_{\text{root}})$$

$$t \models P \wedge Q \quad \Leftrightarrow \quad t \models P \text{ and } t \models Q$$

$$t \models \neg P \quad \Leftrightarrow \quad t \not\models P$$

CTL* Semantics

State Semantics

$$\begin{aligned} t \models p &\Leftrightarrow p \in L(t_{\text{root}}) \\ t \models P \wedge Q &\Leftrightarrow t \models P \text{ and } t \models Q \\ t \models \neg P &\Leftrightarrow t \not\models P \\ t \models \mathbf{E} \varphi &\Leftrightarrow \exists \rho \in \text{Paths}(t). \rho \models \varphi \end{aligned}$$

CTL* Semantics

State Semantics

$$\begin{aligned} t \models p &\Leftrightarrow p \in L(t_{\text{root}}) \\ t \models P \wedge Q &\Leftrightarrow t \models P \text{ and } t \models Q \\ t \models \neg P &\Leftrightarrow t \not\models P \\ t \models \mathbf{E} \varphi &\Leftrightarrow \exists \rho \in \text{Paths}(t). \rho \models \varphi \\ t \models \mathbf{A} \varphi &\Leftrightarrow \forall \rho \in \text{Paths}(t). \rho \models \varphi \end{aligned}$$

CTL* Semantics

State Semantics

$$\begin{aligned}t \models p &\Leftrightarrow p \in L(t_{\text{root}}) \\t \models P \wedge Q &\Leftrightarrow t \models P \text{ and } t \models Q \\t \models \neg P &\Leftrightarrow t \not\models P \\t \models \mathbf{E} \varphi &\Leftrightarrow \exists \rho \in \text{Paths}(t). \rho \models \varphi \\t \models \mathbf{A} \varphi &\Leftrightarrow \forall \rho \in \text{Paths}(t). \rho \models \varphi\end{aligned}$$

Path Semantics

$$\rho \models P \Leftrightarrow \rho_0 \models P$$

CTL* Semantics

State Semantics

$$\begin{aligned}t \models p &\Leftrightarrow p \in L(t_{\text{root}}) \\t \models P \wedge Q &\Leftrightarrow t \models P \text{ and } t \models Q \\t \models \neg P &\Leftrightarrow t \not\models P \\t \models \mathbf{E} \varphi &\Leftrightarrow \exists \rho \in \text{Paths}(t). \rho \models \varphi \\t \models \mathbf{A} \varphi &\Leftrightarrow \forall \rho \in \text{Paths}(t). \rho \models \varphi\end{aligned}$$

Path Semantics

$$\begin{aligned}\rho \models P &\Leftrightarrow \rho_0 \models P \\\rho \models \varphi \wedge \psi &\Leftrightarrow \rho \models \varphi \text{ and } \rho \models \psi \\\rho \models \neg \varphi &\Leftrightarrow \rho \not\models \varphi\end{aligned}$$

CTL* Semantics

State Semantics

$$\begin{aligned}
t \models p &\Leftrightarrow p \in L(t_{\text{root}}) \\
t \models P \wedge Q &\Leftrightarrow t \models P \text{ and } t \models Q \\
t \models \neg P &\Leftrightarrow t \not\models P \\
t \models \mathbf{E} \varphi &\Leftrightarrow \exists \rho \in \text{Paths}(t). \rho \models \varphi \\
t \models \mathbf{A} \varphi &\Leftrightarrow \forall \rho \in \text{Paths}(t). \rho \models \varphi
\end{aligned}$$

Path Semantics

$$\begin{aligned}
\rho \models P &\Leftrightarrow \rho_0 \models P \\
\rho \models \varphi \wedge \psi &\Leftrightarrow \rho \models \varphi \text{ and } \rho \models \psi \\
\rho \models \neg \varphi &\Leftrightarrow \rho \not\models \varphi \\
\rho \models \mathbf{X} \varphi &\Leftrightarrow \rho|_1 \models \varphi
\end{aligned}$$

CTL* Semantics

State Semantics

$$\begin{aligned}
t \models p &\Leftrightarrow p \in L(t_{\text{root}}) \\
t \models P \wedge Q &\Leftrightarrow t \models P \text{ and } t \models Q \\
t \models \neg P &\Leftrightarrow t \not\models P \\
t \models \mathbf{E} \varphi &\Leftrightarrow \exists \rho \in \text{Paths}(t). \rho \models \varphi \\
t \models \mathbf{A} \varphi &\Leftrightarrow \forall \rho \in \text{Paths}(t). \rho \models \varphi
\end{aligned}$$

Path Semantics

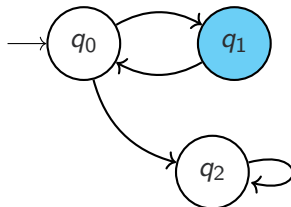
$$\begin{aligned}
\rho \models P &\Leftrightarrow \rho_0 \models P \\
\rho \models \varphi \wedge \psi &\Leftrightarrow \rho \models \varphi \text{ and } \rho \models \psi \\
\rho \models \neg \varphi &\Leftrightarrow \rho \not\models \varphi \\
\rho \models \mathbf{X} \varphi &\Leftrightarrow \rho|_1 \models \varphi \\
\rho \models \varphi \mathbf{U} \psi &\Leftrightarrow \text{There exists an } i \text{ such that } \rho|_i \models \psi \\
&\quad \text{and for all } j < i, \rho|_j \models \varphi
\end{aligned}$$

CTL* Examples

We say a Kripke structure A satisfies a CTL* property P , that is,
 $A \models P$ iff $\text{Tree}(A) \models P$

Given this automaton A :

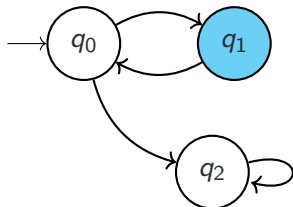
● $A \models \mathbf{E\,G\,F\,}\bullet$?



CTL* Examples

We say a Kripke structure A satisfies a CTL* property P , that is,
 $A \models P$ iff $\text{Tree}(A) \models P$

Given this automaton A :



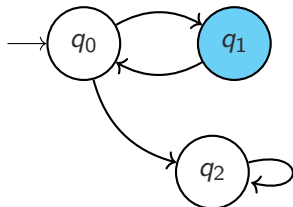
$$\bullet A \models \mathbf{E\,G\,F} \bullet?$$

$$\bullet A \models \mathbf{A\,G\,F} \bullet?$$

CTL* Examples

We say a Kripke structure A satisfies a CTL* property P , that is,
 $A \models P$ iff $\text{Tree}(A) \models P$

Given this automaton A :



● $A \models \mathbf{E\,G\,F}$ ●?

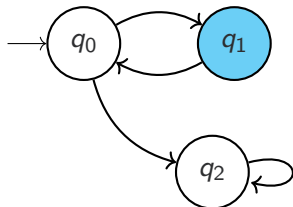
● $A \models \mathbf{A\,G\,F}$ ●?

● $A \models \mathbf{A\,F}$ ●?

CTL* Examples

We say a Kripke structure A satisfies a CTL* property P , that is,
 $A \models P$ iff $\text{Tree}(A) \models P$

Given this automaton A :



$$\bullet A \models \mathbf{E G F} \bullet?$$

$$\bullet A \models \mathbf{A G F} \bullet?$$

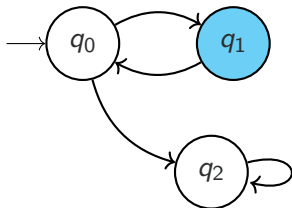
$$\bullet A \models \mathbf{A F} \bullet?$$

$$\bullet A \models \mathbf{A E F} \bullet?$$

CTL* Examples

We say a Kripke structure A satisfies a CTL* property P , that is,
 $A \models P$ iff $\text{Tree}(A) \models P$

Given this automaton A :



- $A \models \mathbf{E\,G\,F}$ ●?
- $A \models \mathbf{A\,G\,F}$ ●?
- $A \models \mathbf{A\,F}$ ●?
- $A \models \mathbf{A\,E\,F}$ ●?
- $A \models \mathbf{E\,A\,F}$ ●?

Simplifying

CTL* is **very expressive** but **very complicated**.

It's also extremely hard to model check, which we'll get to later.

CTL* to CTL

Keep state formulae the same:

- All $p \in \mathcal{P}$ are SFs.
- Given SFs P and Q , $\neg P$ is a SF and $P \wedge Q$ is a SF.
- Given a PF φ , $\mathbf{E}\varphi$ and $\mathbf{A}\varphi$ are SFs.

But we force path formulae to go straight back to state formulae immediately with a temporal operator:

- Given a **SF** P then $\mathbf{X}P$ is a PF.
- Given **SFs** P and Q , $P \mathbf{U} Q$ is a PF.

Examples

Which of the following CTL* formulae are CTL formulae?

- $a \text{ U } (b \text{ U } c)$
- $\mathbf{A} (a \text{ U } c)$
- $\mathbf{X} \mathbf{X} a$
- $\mathbf{X} \mathbf{A} a$
- $\mathbf{A} (a \text{ U } (b \text{ U } c))$
- $\mathbf{A} \mathbf{E} (a \text{ U } b)$
- $\mathbf{E} \mathbf{X} a$
- $\mathbf{X} \mathbf{E} a$

Non-mutual CTL Syntax

Simpler CTL Syntax

A CTL formula is defined as follows:

- All $p \in \mathcal{P}$ are formulae.
- Given formulae P and Q , $\neg P$ is a formula and $P \wedge Q$ is a formula.
- Given a formula P , **EX** P and **AX** P are formulae.
- Given formulae P and Q , **E**($P \mathbf{U} Q$) and **A**($P \mathbf{U} Q$) are formulae.

Simpler CTL Semantics

Semantics are as with CTL*, but can be defined more directly:

Semantics

$$t \models p \quad \Leftrightarrow \quad p \in L(t_{\text{root}})$$

Simpler CTL Semantics

Semantics are as with CTL*, but can be defined more directly:

Semantics

$$\begin{array}{ll} t \models p & \Leftrightarrow p \in L(t_{\text{root}}) \\ t \models P \wedge Q & \Leftrightarrow t \models P \text{ and } t \models Q \\ t \models \neg P & \Leftrightarrow t \not\models P \end{array}$$

Simpler CTL Semantics

Semantics are as with CTL*, but can be defined more directly:

Semantics

$$\begin{array}{ll} t \models p & \Leftrightarrow p \in L(t_{\text{root}}) \\ t \models P \wedge Q & \Leftrightarrow t \models P \text{ and } t \models Q \\ t \models \neg P & \Leftrightarrow t \not\models P \\ t \models \mathbf{EX} P & \Leftrightarrow \exists \rho \in \text{Paths}(t). \rho_1 \models P \end{array}$$

Simpler CTL Semantics

Semantics are as with CTL*, but can be defined more directly:

Semantics

$t \models p$	$\Leftrightarrow p \in L(t_{\text{root}})$
$t \models P \wedge Q$	$\Leftrightarrow t \models P \text{ and } t \models Q$
$t \models \neg P$	$\Leftrightarrow t \not\models P$
$t \models \mathbf{EX} P$	$\Leftrightarrow \exists \rho \in \text{Paths}(t). \rho_1 \models P$
$t \models \mathbf{AX} P$	$\Leftrightarrow \forall \rho \in \text{Paths}(t). \rho_1 \models P$

Simpler CTL Semantics

Semantics are as with CTL*, but can be defined more directly:

Semantics

$$\begin{array}{ll} t \models p & \Leftrightarrow p \in L(t_{\text{root}}) \\ t \models P \wedge Q & \Leftrightarrow t \models P \text{ and } t \models Q \\ t \models \neg P & \Leftrightarrow t \not\models P \\ t \models \mathbf{EX} P & \Leftrightarrow \exists \rho \in \text{Paths}(t). \rho_1 \models P \\ t \models \mathbf{AX} P & \Leftrightarrow \forall \rho \in \text{Paths}(t). \rho_1 \models P \\ t \models \mathbf{A}(P \mathbf{U} Q) & \Leftrightarrow \forall \rho \in \text{Paths}(t), \text{ there } \exists \text{ an } i \text{ such that:} \\ & \rho_i \models Q \text{ and } \forall j < i. \rho_j \models P \end{array}$$

Simpler CTL Semantics

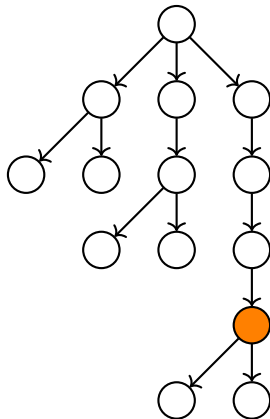
Semantics are as with CTL*, but can be defined more directly:

Semantics

$t \models p$	$\Leftrightarrow$	$p \in L(t_{\text{root}})$
$t \models P \wedge Q$	$\Leftrightarrow$	$t \models P$ and $t \models Q$
$t \models \neg P$	$\Leftrightarrow$	$t \not\models P$
$t \models \mathbf{EX} P$	$\Leftrightarrow$	$\exists \rho \in \text{Paths}(t). \rho_1 \models P$
$t \models \mathbf{AX} P$	$\Leftrightarrow$	$\forall \rho \in \text{Paths}(t). \rho_1 \models P$
$t \models \mathbf{A}(P \mathbf{U} Q)$	$\Leftrightarrow$	$\forall \rho \in \text{Paths}(t)$, there $\exists$ an i such that: $\rho_i \models Q$ and $\forall j < i. \rho_j \models P$
$t \models \mathbf{E}(P \mathbf{U} Q)$	$\Leftrightarrow$	$\exists \rho \in \text{Paths}(t)$ and an i such that: $\rho_i \models Q$ and $\forall j < i. \rho_j \models P$

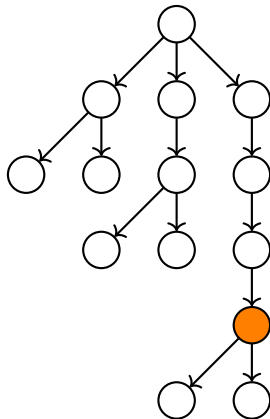
Derived Operators

Define **EF**●:



Derived Operators

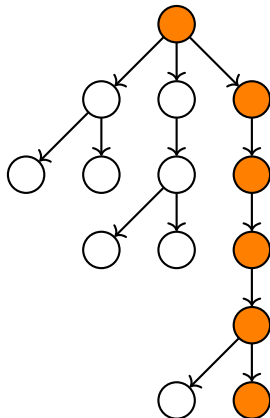
Define **EF** ●:



E(True **U** ●)

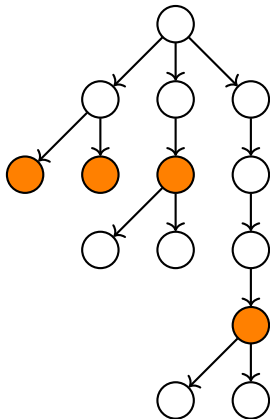
Derived Operators

Define **EG** ●:



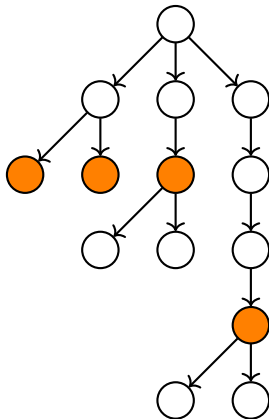
Derived Operators

Define **AF**●:



Derived Operators

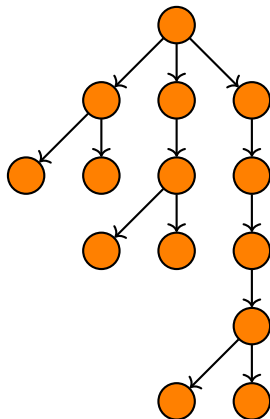
Define **AF**●:



A(True **U** ●)

Derived Operators

Define **AG**●:



EF

Bibliography

- Huth/Ryan: Logic in Computer Science, Section 3.2 and 3.4
- Bayer/Katoen: Principles of Model Checking Sections 5.1 and 6.2

Weekly Feedback

I would appreciate any comments/suggestions/requests you have on this week's lectures (and tutorials).



<https://forms.office.com/r/sCi2xjavrS>